Kubernetes Hands-On Demo: Deploying a Spring Boot Application with MySQL

In this practical, we deploy a Spring Boot application and a MySQL database on a local Kubernetes cluster using Minikube. The demo covers configuration management, Deployments, Services, replicas, scaling, self-healing, rolling updates and rollback.

1. Application Architecture

The application contains two workloads:

- **Spring Boot Notes API:** Serves HTTP requests and reads/writes notes to MySQL.
- **MySQL:** Stores the notes.

Kubernetes will run them using the following resources:

Resource	Purpose
ConfigMap	Stores the database name and JDBC connection URL
Secret	Stores database credentials
MySQL Deployment	Maintains one MySQL Pod
`mysql-service` (ClusterIP)	Provides a stable, internal address for MySQL
Notes App Deployment	Maintains two Spring Boot Pods initially
`notes-service` (NodePort)	Exposes the Spring Boot API on port `30080` of the Minikube node

Request flow:

```
Client (Ubuntu VM)
       |
```

```
        v
Minikube Node IP:30080
        |
        v
notes-service (NodePort)
        |
        +--------+--------+
        |                 |
        v                 v
  Notes Pod 1       Notes Pod 2
        |                 |
        +--------+--------+
                 |
                 v
     mysql-service (ClusterIP)
                 |
                 v
            MySQL Pod
```

The existing Spring Boot project is assumed to include a `Dockerfile`, `pom.xml` and the `/notes`, `/whoami` and `/version` endpoints used in this practical.

2. Prepare the Ubuntu Environment

The Minikube cluster runs **inside the Ubuntu Multipass VM**, rather than directly on the Mac.

On the Mac, transfer the project into Ubuntu:

```
multipass transfer -r ~/Desktop/kubernetesDemo devops:/home/ubuntu/
```

Open the Ubuntu shell:

```
multipass shell devops
```

Inside Ubuntu, navigate to the project:

```
cd /home/ubuntu/kubernetesDemo
ls
```

The project root should contain the application files, including `Dockerfile`, `pom.xml` and `src/`.

Verify Minikube

```
minikube status
```

If the cluster is stopped:

```
minikube start --driver=docker
```

Confirm that the node is ready:

```
kubectl get nodes
```

The Minikube node should eventually show `Ready`.

3. Build the Spring Boot Image in Minikube

For this local demo, the application image is built directly in Minikube. It does not need to be published to Docker Hub.

Run from `/home/ubuntu/kubernetesDemo`:

```
minikube image build -t notes-app:v1 .
```

Verify that the image is available:

```
minikube image ls | grep notes-app
```

The image `notes-app:v1` will be referenced by the Spring Boot Deployment.

Create a directory for Kubernetes manifests:

```
mkdir -p k8s
```

The project structure will include:

```
kubernetesDemo/
├── Dockerfile
├── pom.xml
├── src/
└── k8s/
    ├── 01-config.yaml
    ├── 02-secret.yaml
    ├── 03-mysql.yaml
    └── 04-app.yaml
```

4. Create the ConfigMap

A **ConfigMap** stores non-sensitive configuration separately from the container image. In this example, it contains the database name and JDBC URL.

Create `k8s/01-config.yaml`:

```
cat > k8s/01-config.yaml <<'EOF'
apiVersion: v1
kind: ConfigMap
metadata:
  name: notes-config

data:
  MYSQL_DATABASE: notesdb
  DB_URL: "jdbc:mysql://mysql-service:3306/notesdb?allowPublicKeyRetrieval=true&useSSL=fa
lse"
EOF
```

Apply the manifest:

```
kubectl apply -f k8s/01-config.yaml
```

Check the resource:

```
kubectl get configmaps
kubectl describe configmap notes-config
```

The ConfigMap has **two data entries**: `MYSQL_DATABASE` and `DB_URL`.

The JDBC URL uses `mysql-service`, the Kubernetes Service name, rather than the IP address of an individual MySQL Pod. The Service provides a stable address even if the underlying Pod is replaced.

5. Create the Secret

A **Secret** stores configuration intended to be handled as sensitive data, such as usernames and passwords. The demo uses simple example credentials, which should not be used for a real deployment.

Create `k8s/02-secret.yaml`:

```
cat > k8s/02-secret.yaml <<'EOF'
apiVersion: v1
kind: Secret
metadata:
  name: mysql-secret

type: Opaque

stringData:
  MYSQL_ROOT_PASSWORD: rootpass
  MYSQL_USER: notesuser
  MYSQL_PASSWORD: notespass
EOF
```

Apply and inspect it:

```
kubectl apply -f k8s/02-secret.yaml
kubectl get secrets
kubectl describe secret mysql-secret
```

Key fields:

- `type: Opaque` specifies a generic Secret with key-value data.

- `stringData` lets the manifest provide plain-text values that Kubernetes processes into Secret data.
- `kubectl describe secret` displays the keys and their sizes, but not their values.

Neither the ConfigMap nor the Secret runs a container. Their values will be injected into Pods as environment variables.

6. Deploy MySQL and Its ClusterIP Service

MySQL needs a **Deployment** to maintain its Pod and a **Service** to make the database reachable within the cluster.

Create `k8s/03-mysql.yaml`:

```
cat > k8s/03-mysql.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql

spec:
  replicas: 1
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:8.4
          ports:
            - containerPort: 3306
          env:
            - name: MYSQL_DATABASE
              valueFrom:
                configMapKeyRef:
                  name: notes-config
                  key: MYSQL_DATABASE
            - name: MYSQL_USER
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
                  key: MYSQL_USER
            - name: MYSQL_PASSWORD
```

```
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
                  key: MYSQL_PASSWORD
            - name: MYSQL_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
                  key: MYSQL_ROOT_PASSWORD
          readinessProbe:
            tcpSocket:
              port: 3306
            initialDelaySeconds: 10
            periodSeconds: 5
          resources:
            requests:
              cpu: "250m"
              memory: "512Mi"
            limits:
              cpu: "250m"
              memory: "512Mi"
---
apiVersion: v1
kind: Service
metadata:
  name: mysql-service
spec:
  type: ClusterIP
  selector:
    app: mysql
  ports:
    - port: 3306
      targetPort: 3306
EOF
```

The `---` separator allows one YAML file to define more than one Kubernetes resource.

Important parts of the MySQL manifest

Field	Meaning
`kind: Deployment`	Declares a Deployment controller
`replicas: 1`	Keeps one MySQL Pod running
`selector.matchLabels`	Identifies the Pods managed by the Deployment
`template`	Defines how Kubernetes creates each Pod

Field	Meaning
`image: mysql:8.4`	Selects the MySQL container image
`containerPort: 3306`	Declares the MySQL container port
`valueFrom`	Reads environment values from the ConfigMap or Secret
`readinessProbe`	Checks whether port `3306` is accepting TCP connections
`resources.requests`	Specifies CPU and memory requested for scheduling
`resources.limits`	Sets CPU and memory limits for the container
`type: ClusterIP`	Makes the database Service accessible within the cluster

The readiness probe begins after **10 seconds** and repeats every **5 seconds**. This TCP check indicates that the port can accept connections; it does not execute a database query.

Why ClusterIP for MySQL?

The Spring Boot Pods need to reach MySQL, but the database does not need to be exposed through a NodePort. `ClusterIP` supplies an internal Service address and routes requests to the matching MySQL Pod.

Apply and verify

```
kubectl apply -f k8s/03-mysql.yaml
kubectl get pods
kubectl get pods -w
```

The Pod may initially show `ContainerCreating`. Once it shows `1/1 Running`, press `Ctrl + C` to exit watch mode.

Verify the rollout:

```
kubectl rollout status deployment/mysql --timeout=240s
```

Inspect the resources:

```
kubectl get deployments
kubectl get replicasets
```

```
kubectl get pods
kubectl get services
```

The Deployment manages a ReplicaSet, and the ReplicaSet maintains the required Pod count:

```
Deployment: mysql
       |
       v
    ReplicaSet
       |
       v
    MySQL Pod
       |
       v
 MySQL Container
```

The Service can be reached using the DNS name `mysql-service` inside the cluster. It is not tied to the IP address of any one Pod.

7. Deploy Spring Boot and Its NodePort Service

The Spring Boot application starts with **two replicas**, allowing the demo to show traffic routing, scaling and self-healing.

Create `k8s/04-app.yaml`:

```
cat > k8s/04-app.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: notes-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: notes-app
  template:
    metadata:
      labels:
        app: notes-app
    spec:
      containers:
        - name: notes-app
```

```
          image: notes-app:v1
          imagePullPolicy: Never
          ports:
            - containerPort: 8080
          env:
            - name: DB_URL
              valueFrom:
                configMapKeyRef:
                  name: notes-config
                  key: DB_URL
            - name: DB_USERNAME
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
                  key: MYSQL_USER
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
                  key: MYSQL_PASSWORD
            - name: APP_VERSION
              value: "v1"
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
---
apiVersion: v1
kind: Service
metadata:
  name: notes-service
spec:
  type: NodePort
  selector:
    app: notes-app
  ports:
    - port: 8080
      targetPort: 8080
      nodePort: 30080
EOF
```

Key configuration

- `replicas: 2` — Maintains two interchangeable Spring Boot Pods.
- `image: notes-app:v1` — Uses the application image built in Minikube.

- `imagePullPolicy: Never` — Uses the locally available image rather than pulling it from a registry.
- `DB_URL`, `DB_USERNAME`, `DB_PASSWORD` — Receive configuration from the ConfigMap and Secret.
- `APP_VERSION: v1` — Supplies a version value that can be changed during the rolling-update demo.
- `type: NodePort` — Makes the application reachable through the Minikube node on port `30080`.

Apply and verify

```
kubectl apply -f k8s/04-app.yaml
kubectl get deployments
kubectl get replicasets
kubectl get pods
kubectl get services
```

The cluster should contain one MySQL Pod and two Notes App Pods. The Services should include:

```
mysql-service   ClusterIP   3306/TCP
notes-service   NodePort    8080:30080/TCP
```

The two Services have different roles: `mysql-service` handles internal database communication, while `notes-service` exposes the HTTP API through the node.

8. Access the Application Through NodePort

Get the Minikube node IP **inside Ubuntu**:

```
minikube ip
```

Store it in a shell variable:

```
MINIKUBE_IP=$(minikube ip)
```

Create a note:

```
curl -X POST "http://$MINIKUBE_IP:30080/notes?text=Hello-Kubernetes"
```

Example response:

```
{"id":1,"text":"Hello-Kubernetes"}
```

Retrieve the notes:

```
curl "http://$MINIKUBE_IP:30080/notes"
```

The request passes through the NodePort Service to a Spring Boot Pod, which accesses MySQL through its ClusterIP Service.

Important: These commands assume the application and database have started successfully. Port `30080` is being accessed from the Ubuntu VM running Minikube; access directly from the Mac may require a different network setup.

9. Observe Service Routing Across Pods

The `/whoami` endpoint returns the identity of the application Pod that handled the request.

```
curl "http://$MINIKUBE_IP:30080/whoami"
```

Example output:

```
Handled by: notes-app-6c769d5b67-zjbx2
```

Repeat the call:

```
for i in {1..10}; do
  curl -s "http://$MINIKUBE_IP:30080/whoami"
  echo
done
```

Responses **may** show different Pod names. The Service routes traffic to eligible backend Pods; the individual requests are not guaranteed to alternate between Pods.

The client only needs the Service address, not the IP address of each Pod.

10. Scale the Spring Boot Deployment

Check the current application Pods:

```
kubectl get pods -l app=notes-app
```

Increase the desired replica count from two to three:

```
kubectl scale deployment notes-app --replicas=3
```

Verify:

```
kubectl get pods -l app=notes-app
```

Kubernetes compares **desired state** with **actual state**:

```
Before scaling: desired = 2, actual = 2
After command: desired = 3, actual = 2
After reconciliation: desired = 3, actual = 3
```

Repeat the routing test:

```
for i in {1..15}; do
  curl -s "http://$MINIKUBE_IP:30080/whoami"
```

```
  echo
done
```

Requests may now be handled by any of the three available Pods.

Note: `kubectl scale` changes the live Deployment. The original `04-app.yaml` still declares `replicas: 2`; applying that file again can restore the old replica count. For a lasting declarative change, update the YAML as well.

11. Demonstrate Self-Healing

List the Notes App Pods:

```
kubectl get pods -l app=notes-app
```

Select one running Notes App Pod and delete it:

```
POD_TO_DELETE=$(kubectl get pods -l app=notes-app -o jsonpath='{.items[0].metadata.nam
e}')
kubectl delete pod "$POD_TO_DELETE"
```

The first command stores the name of an existing application Pod; the second deletes that Pod.

Watch the remaining Pods:

```
kubectl get pods -l app=notes-app -w
```

The ReplicaSet creates a replacement because the Deployment still requires three replicas.

```
Desired replicas = 3
One Pod deleted
Actual replicas = 2
ReplicaSet creates another Pod
Actual replicas = 3
```

This behavior illustrates Kubernetes **reconciliation and self-healing**: a controller works to restore the desired number of Pods without a manual `kubectl run` command.

12. Perform a Rolling Update

The deployment initially sets `APP_VERSION=v1`. The `/version` endpoint reads this value.

Check the current version:

```
curl "http://$MINIKUBE_IP:30080/version"
```

Example response:

```
Application version: v1
```

Update the Deployment environment variable:

```
kubectl set env deployment/notes-app APP_VERSION=v2
```

Inspect the rollout:

```
kubectl get pods
kubectl rollout status deployment/notes-app
kubectl get replicasets
```

Verify the new version:

```
curl "http://$MINIKUBE_IP:30080/version"
```

Expected after the rollout finishes:

```
Application version: v2
```

Changing an environment variable inside the **Pod template** causes Kubernetes to create a new ReplicaSet and replace the old Pods through a rolling update.

```
notes-app Deployment
        |
        +-- Old ReplicaSet -> old v1 Pods (scaled down)
        |
        +-- New ReplicaSet -> new v2 Pods (scaled up)
```

This demo changes an environment variable rather than the application image. A normal application release would typically use a new image tag, such as `notes-app:v2`, but the Deployment rollout mechanism is the same.

Note: `kubectl set env` changes the live Deployment, not the saved YAML. Update `04-app.yaml` if `v2` should become the declared configuration.

13. Roll Back the Deployment

View the rollout history:

```
kubectl rollout history deployment/notes-app
```

Undo the latest rollout:

```
kubectl rollout undo deployment/notes-app
```

Wait for it to finish:

```
kubectl rollout status deployment/notes-app
```

Check the version again:

```
curl "http://$MINIKUBE_IP:30080/version"
```

After the rollback, the application should report `v1` again, provided the previous revision was the original `v1` configuration.

14. Why a Deployment Is Not Enough for Production MySQL

The demo uses a MySQL Deployment **without persistent storage**. This keeps the initial setup small, but it is not a safe way to store important database data.

If the MySQL Pod is deleted and recreated, data held only in the old container filesystem can be lost. The new Pod may start with a fresh database.

```
Old MySQL Pod deleted
        |
        v
Container-local database files lost
        |
        v
Replacement MySQL Pod starts
        |
        v
Previously stored notes may be missing
```

Optional destructive demonstration (only if test data may be discarded):

```
MYSQL_POD=$(kubectl get pods -l app=mysql -o jsonpath='{.items[0].metadata.name}')
kubectl delete pod "$MYSQL_POD"
```

The MySQL Deployment will create a replacement Pod, but that does not guarantee the original database contents survive. During recovery, the Notes API may also return errors until the database is ready and the application reconnects.

Persistent data requires appropriate storage, such as a **PersistentVolume** used through a claim. A **StatefulSet** is also commonly used for stateful workloads that need stable identities and storage behavior.

Deployment, StatefulSet and DaemonSet

Controller	Suitable use
Deployment	Interchangeable, usually stateless application Pods, such as the Notes API
StatefulSet	Workloads requiring stable identity or persistent storage, such as many database deployments
DaemonSet	Runs a Pod on each eligible node, useful for node-level agents rather than this Notes API or MySQL database

A DaemonSet is not needed here because neither workload should automatically run one copy on every node.

15. Summary

This practical demonstrates the core Kubernetes workflow:

1. Transfer the Spring Boot project into the Ubuntu VM running Minikube.
2. Build the application image inside Minikube.
3. Create a ConfigMap and Secret for database configuration.
4. Deploy MySQL with a ClusterIP Service.
5. Deploy two Spring Boot Pods with a NodePort Service.
6. Test API requests and observe routing to different Pods.
7. Scale the Deployment to three replicas.
8. Delete an application Pod and observe self-healing.
9. Trigger a rolling update by changing a Pod-template environment variable.
10. Roll back to the previous Deployment revision.
11. Understand why a real database needs persistent storage and careful state management.

Core concept: Kubernetes continually works to make the actual state of the cluster match the desired state declared by its controllers.